TrackerApp: Posicionamento *Indoor* por BLE com Nós ESP32, Telemetria MQTT em Pilha Dupla IPv6/IPv4 e Painéis em Tempo Real

Camila Cirne[1], Anderson Gabriel[1], Luís Filipe Macedo dos Santos[1], [PENDENTE: demais integrantes][1]

[1]CESAR School – Recife, PE – Brasil
camilacirne@hotmail.com, lfms2@cesar.org.br

Abstract. *This report describes TrackerApp, an indoor positioning system built for the Embedded Systems course. Fixed ESP32 boards act as Bluetooth Low Energy (BLE) beacons, and a collector ESP32 or a smartphone measures their received signal strength (RSSI). A log-distance path loss model turns RSSI into distance, and least-squares multilateration over $N \geq 3$ anchors yields a 2D position. The collector publishes continuous telemetry over MQTT to an Eclipse Mosquitto broker using an IPv6-first connection with automatic IPv4 fallback, which detects an IPv6 infrastructure failure and recovers without rebooting the board. A FastAPI service stores the data in PostgreSQL, Grafana shows historical panels (peak hours, density, occupancy and dwell time), and a Next.js map shows live positions pushed through Server-Sent Events.*

Resumo. *Este relatório descreve o TrackerApp, um sistema de posicionamento indoor desenvolvido na disciplina de Sistemas Embarcados. Placas ESP32 fixas atuam como beacons Bluetooth Low Energy (BLE), e um ESP32 coletor ou um smartphone mede a intensidade do sinal recebido (RSSI). Um modelo log-distância de perda de percurso converte RSSI em distância, e a multilateração por mínimos quadrados sobre $N \geq 3$ âncoras fornece uma posição 2D. O coletor publica telemetria contínua via MQTT num broker Eclipse Mosquitto, com conexão que tenta IPv6 primeiro e recai automaticamente para IPv4, detectando a falha da infraestrutura IPv6 e se recuperando sem reiniciar a placa. Um serviço FastAPI grava os dados em PostgreSQL, o Grafana exibe painéis históricos (horário de pico, densidade, ocupação e permanência) e um mapa em Next.js mostra as posições ao vivo, recebidas por Server-Sent Events.*

1 Introdução

Sistemas de navegação por satélite (GNSS) não funcionam bem dentro de edificações: o sinal chega atenuado e refletido, e o erro passa facilmente de dezenas de metros. Saber onde pessoas e objetos estão em ambientes internos — salas de aula, laboratórios, corredores — exige outra infraestrutura. Entre as alternativas (Wi-Fi, UWB, visão computacional, BLE), o Bluetooth Low Energy se destaca pelo baixo custo dos nós e pela presença em praticamente todo *smartphone* [Zafari et al. 2019].

O **TrackerApp** explora essa alternativa com uma infraestrutura de baixo custo: placas ESP32 fixas anunciam um identificador BLE, e um observador (outro ESP32 ou um celular) mede o RSSI de cada uma, estima a distância e calcula a própria posição. Os dados seguem por MQTT e HTTP para um *backend* que alimenta painéis em tempo real e históricos, e também formam um *dataset* rotulado (*fingerprinting*) para o treinamento posterior de modelos de aprendizado de máquina, abordagem que associa o vetor de RSSI medido num ponto à posição conhecida desse ponto [Bahl e Padmanabhan 2000, Faragher e Harle 2015].

Tema do projeto. O projeto se enquadra no tema **Computação**, um dos cinco temas definidos pela disciplina: trata de computação ubíqua e Internet das Coisas aplicadas à localização em ambientes internos, à análise de ocupação de espaços e à coleta de dados para aprendizado de máquina.

Objetivos. O objetivo geral é construir um sistema embarcado de ponta a ponta para posicionamento *indoor*. Os objetivos específicos, alinhados aos requisitos da disciplina, são:

1. desenvolver o *firmware* do ESP32 nos papéis de *beacon* e de coletor de RSSI, com um protocolo de anúncio compartilhado com os aplicativos iOS e Android;
2. transmitir a telemetria do coletor por **MQTT** a um *broker*;
3. usar **IPv6** na comunicação, com detecção de falha da infraestrutura e retorno automático para IPv4;
4. coletar e armazenar os dados (PostgreSQL) por meio de uma API;
5. exibir os dados em **painéis em tempo real** (mapa da planta e Grafana).

O restante do relatório está organizado assim: a Seção 2 apresenta os conceitos usados; a Seção 3, a arquitetura e as decisões de implementação; a Seção 4, os resultados obtidos; e a Seção 5, as conclusões e os trabalhos futuros. Os principais códigos estão nos apêndices.

2 Fundamentação Teórica

2.1 Bluetooth Low Energy e RSSI

No BLE, um dispositivo pode apenas anunciar pacotes periódicos (*advertising*) sem estabelecer conexão. No anúncio legado, a carga útil é limitada a 31 bytes [Bluetooth SIG 2023]. Todo receptor que escuta um anúncio obtém o *Received Signal Strength Indicator* (RSSI), em dBm, que diminui com a distância, mas também varia com obstáculos, reflexões, orientação da antena e o corpo de quem segura o aparelho.

2.2 Modelo log-distância de perda de percurso

A relação entre RSSI e distância é aproximada pelo modelo log-distância [Rappaport 2002]:

$$\mathrm{RSSI}(d) = \mathrm{RSSI}_0 - 10\,n\,\log_{10}\!\left(\frac{d}{d_0}\right) \quad \Longrightarrow \quad d = d_0 \cdot 10^{\frac{\mathrm{RSSI}_0 - \mathrm{RSSI}}{10\,n}}, \tag{1}$$

em que RSSI_0 é o RSSI medido à distância de referência $d_0 = 1\,\mathrm{m}$ e n é o expoente de perda (2 no espaço livre; tipicamente entre 2 e 4 em ambientes internos). O projeto usa $\mathrm{RSSI}_0 = -59\,\mathrm{dBm}$ e $n = 2{,}0$ como valores nominais, ainda a calibrar no ambiente real. Para reduzir o ruído, o RSSI de cada âncora passa por suavização exponencial ($\alpha = 0{,}15$) antes da conversão.

2.3 Multilateração por mínimos quadrados

Dadas N âncoras em posições conhecidas (x_i, y_i) e distâncias estimadas d_i, a posição (x, y) satisfaz $(x - x_i)^2 + (y - y_i)^2 = d_i^2$. Subtraindo a equação da primeira âncora das demais, obtém-se um sistema linear $A\,\mathbf{p} = \mathbf{b}$, com $\mathbf{p} = (x, y)^T$ e, para $i = 2, \ldots, N$:

$$A_i = [\,2(x_i - x_1),\; 2(y_i - y_1)\,], \qquad b_i = d_1^2 - d_i^2 + x_i^2 - x_1^2 + y_i^2 - y_1^2. \tag{2}$$

Com $N \geq 3$ âncoras não colineares, a solução por mínimos quadrados é $\mathbf{p} = (A^T A)^{-1} A^T \mathbf{b}$. Para $N = 3$ ela coincide com a trilateração em forma fechada. A incerteza reportada é o erro quadrático médio entre as distâncias medidas e as distâncias da posição calculada a cada âncora.

2.4 MQTT

O MQTT é um protocolo de mensagens *publish/subscribe* sobre TCP, padronizado pela OASIS [OASIS 2014]. Clientes publicam mensagens em tópicos hierárquicos (por exemplo, `tracker/A1B2C3/sightings`), e um *broker* as entrega a quem assinou o tópico, inclusive com curingas (`tracker/+/sightings`). O cabeçalho mínimo de 2 bytes e a conexão persistente tornam o protocolo adequado a dispositivos com poucos recursos, como o ESP32. O projeto usa o *broker* Eclipse Mosquitto [Light 2017].

2.5 IPv6 e pilha dupla

O IPv6 [Deering e Hinden 2017] amplia o espaço de endereçamento para 128 bits e elimina a necessidade de NAT para dispositivos IoT. Na transição, redes e servidores costumam operar em pilha dupla (IPv4 e IPv6). O mecanismo *Happy Eyeballs* [Schinazi e Pauly 2017] recomenda preferir IPv6 e recorrer ao IPv4 quando o IPv6 não responde em pouco tempo, para que uma falha do caminho IPv6 não se torne uma falha do serviço. O *firmware* do projeto aplica uma versão sequencial dessa ideia (Seção 3.5).

3 Metodologia

3.1 Arquitetura geral

A Figura 1 mostra os componentes do sistema. O código fica num único repositório (*monorepo*) organizado conforme a estrutura pedida pela disciplina: `esp32-esp8266/` (*firmware*), `applications/` (iOS, Android, API e *web*), `docs/` (este relatório) e `schematics/` (diagramas de circuito).

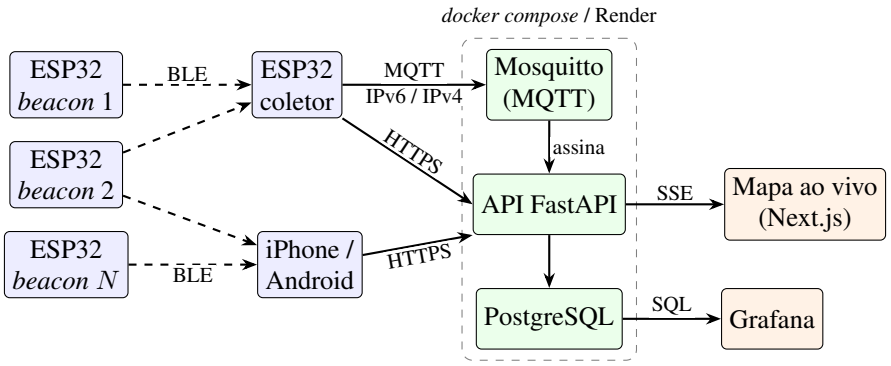


Figura 1. Arquitetura do TrackerApp. Linhas tracejadas: anúncios BLE; linhas cheias: rede IP.

O fluxo de dados tem três caminhos, cada um com um propósito:

- **Telemetria contínua (MQTT):** o ESP32 coletor publica uma mensagem por anúncio BLE recebido; a API assina os tópicos e grava cada leitura na tabela `sightings`.
- **Posição calculada (HTTP):** o celular observador calcula a própria posição a cada segundo e a envia em `POST /fixes`; a API a repassa ao mapa por SSE.
- ***Dataset* rotulado (HTTP):** gravações feitas num ponto de posição conhecida sobem em lote para `POST /fingerprints`, base de treino dos modelos de ML.

3.2 Hardware

Cada nó é uma placa **ESP32 DevKit** (módulo ESP32-WROOM-32, núcleo duplo Xtensa LX6 de 240 MHz, 520 KB de SRAM, Wi-Fi 802.11 b/g/n e Bluetooth 4.2/BLE) [Espressif 2023], usando a antena de PCB do próprio módulo. Os nós são alimentados pela porta USB (carregador ou *power bank*), que também serve para gravação e para o console serial (115 200 bps). O mesmo binário roda em todas as placas; o papel (*beacon* ou coletor) e o identificador ficam na memória não volátil (NVS) e são definidos pelo console serial. O diagrama de circuito do nó está na Figura 2 e em `schematics/`.

3.3 *Firmware* do ESP32

O *firmware* foi escrito em C++17 sobre o *framework* Arduino-ESP32 (plataforma `espressif32@8.0.0`), com PlatformIO, e usa as bibliotecas NimBLE-Arduino 2.5.1

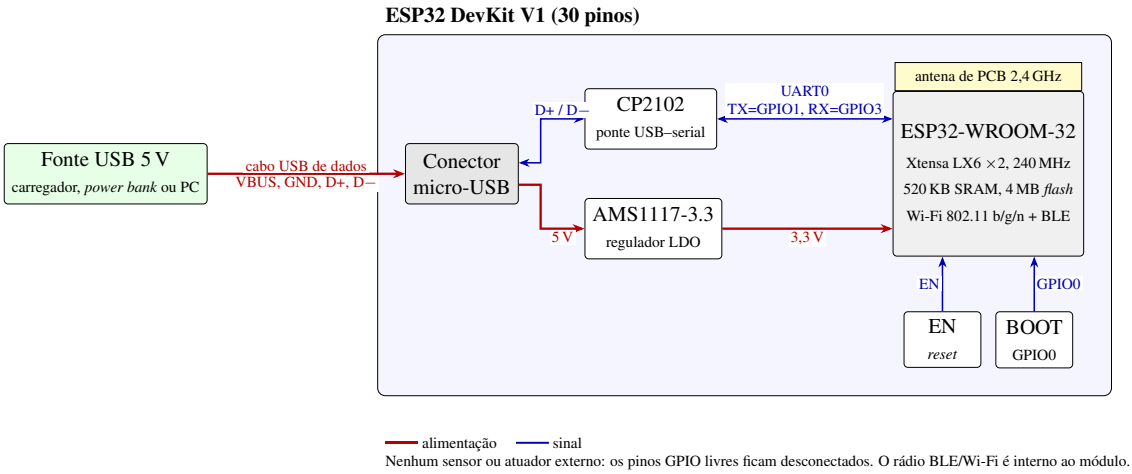


Figura 2. Diagrama do nó ESP32 (*beacon* ou coletor): a placa é alimentada e gravada pela USB e usa a antena de PCB do módulo. Também disponível em `schematics/esp32-node/`.

(pilha BLE), espMqttClient 1.7.3 (cliente MQTT) e AsyncTCP 3.5.0. As versões são fixas no `platformio.ini` para garantir compilações reprodutíveis.

O código segue uma separação em camadas:

- `lib/domain/`: lógica pura, sem dependência de *hardware* (identificador do *beacon*, *parser* de comandos, rótulos, formato dos registros, filas, cargas JSON e a estratégia de pilha dupla). É testada no computador, sem placa;
- `src/hal/`: adaptadores de *hardware* (rádio BLE, relógio, NVS, cliente MQTT, *upload* HTTP), atrás de interfaces;
- `src/app/`: papéis (*beacon* e coletor), tarefa de gravação e um supervisor que troca de papel e despacha eventos. Os objetos têm alocação estática; nada é alocado em regime permanente.

Papel *beacon*. Anuncia pacotes não conectáveis a cada 100 ms com 0 dBm de potência (ambos configuráveis). O pacote tem exatamente 31 bytes: *flags* (3), o UUID de serviço de 128 bits do projeto (18) e o nome local com o identificador de 8 caracteres hexadecimais (10). O mesmo protocolo é usado pelos aplicativos iOS e Android, de modo que celulares também podem servir de âncora.

Papel coletor. Faz varredura contínua (janela de 99 ms a cada 100 ms) e só aceita anúncios com o UUID do projeto e nome de 8 caracteres hexadecimais; a identidade é o ID, nunca o endereço MAC, que pode mudar. No modo ocioso, imprime uma linha por *beacon* por segundo (RSSI e taxa de anúncios); no modo de gravação (`label`, `rec start`, `rec stop`), imprime uma linha por anúncio com o rótulo do ponto (sessão, ponto, x, y) e ao final um resumo por *beacon*. Cada leitura também é publicada por MQTT e, se gravando, enfileirada para `POST /fingerprints` em lotes de até 20 registros ou a cada 5 s.

Configuração. Credenciais e endereços ficam em `include/config.h`, ignorado pelo Git (há um `config.example.h` versionado). Comandos seriais (`id`, `role`, `power`, `interval`, `config save|reset`) gravam na NVS, que tem precedência sobre os valores de compilação.

3.4 Comunicação MQTT

O coletor publica no tópico `<prefixo>/<node_id>/sightings` (por exemplo, `tracker/A1B2C3/sightings`), com QoS 0 e sem retenção, uma mensagem JSON por anúncio:

```
{"observerID":"A1B2C3","anchorID":"9F3C1A02","rssi":-67}
```

O QoS 0 foi escolhido porque cada leitura é logo substituída pela seguinte: perder uma amostra isolada não prejudica a visualização, e evitar confirmações reduz o tráfego disputado com o BLE (o ESP32 compartilha um único rádio de 2,4 GHz entre Wi-Fi e Bluetooth). A placa não envia horário, pois não tem relógio sincronizado; o servidor usa o instante de recepção.

No servidor, o Mosquitto 2.0 roda num contêiner, e o módulo `mqtt_consumer.py` da API assina `tracker/+/sightings` numa *thread* própria (`loop_start`), converte o RSSI em distância com a Equação 1 e grava a leitura. Mensagens malformadas são registradas e descartadas sem derrubar o assinante (Apêndice C).

3.5 IPv6 com detecção de falha e retorno para IPv4

Quando o endereço IPv6 do *broker* é configurado (`MQTT_HOST_V6`), o *firmware* ativa o IPv6 na interface Wi-Fi (SLAAC) e, a cada nova tentativa de conexão, tenta IPv6 primeiro. Se a conexão não for estabelecida em 5 s, a estratégia troca para o endereço IPv4 sem reiniciar a placa e emite um evento de aviso na serial. Uma nova tentativa (Wi-Fi reconectado ou conexão MQTT perdida) volta a tentar IPv6, já que uma queda isolada não significa que o caminho IPv6 deixou de existir. Sem endereço IPv6 configurado, a etapa é pulada. A Figura 3 resume a máquina de estados.

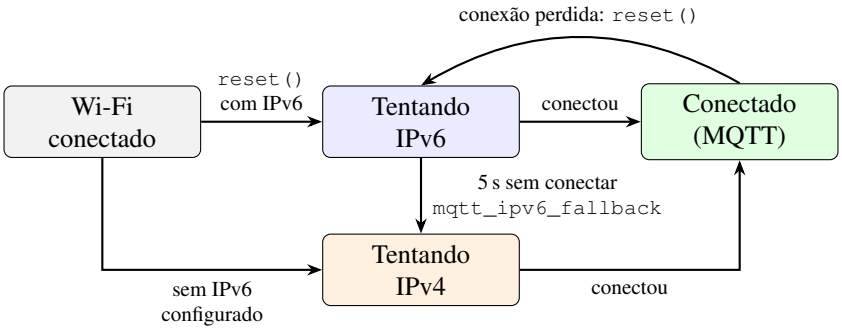


Figura 3. Estratégia de conexão em pilha dupla do publicador MQTT.

A decisão de qual família usar fica numa classe pura

(dual_stack_strategy.h, Apêndice A), sem cabeçalhos de rede, o que permite testar a temporização no computador; os *sockets* ficam no adaptador esp_mqtt_publisher.cpp. Os eventos mqtt_dual_stack, mqtt_connected ("*connected over IPv6*" ou "*over IPv4*") e mqtt_ipv6_fallback na serial demonstram a detecção.

3.6 *Backend* e armazenamento

A API foi escrita em Python com FastAPI [Ramírez 2018] e SQLAlchemy, sobre PostgreSQL 16. Os *endpoints* de escrita (POST /sightings, /fixes e /fingerprints) exigem o cabeçalho X-API-Key; os de leitura (GET /health, /fixes/recent e /fixes/stream) são abertos. A distância é sempre recalculada no servidor a partir do RSSI, para que exista uma única fórmula, independente do cliente. As tabelas são:

- sightings: observador, âncora, RSSI, distância, instantes de leitura e recepção;
- fixes: posição (x, y) calculada, incerteza e número de âncoras;
- fingerprint_records: leituras rotuladas com a posição real do ponto e, nos celulares, orientação, altitude relativa e estado de movimento.

Localmente, um docker-compose.yml sobe PostgreSQL, Mosquitto, API, Grafana e o mapa (Apêndice F). Em produção, API, Grafana e banco rodam como serviços no Render.

3.7 Painéis (*dashboards*)

Mapa ao vivo. Aplicação Next.js/TypeScript que desenha a planta em SVG, com um ponto por observador, rastro das últimas posições, halo proporcional à incerteza e uma camada de calor (células de 0,5 m) que pode ser ligada e desligada. As posições chegam por *Server-Sent Events* (GET /fixes/stream): ao conectar, o cliente recebe um retrato das posições recentes e, depois, um evento por posição nova, sem consultas periódicas. O navegador reconecta sozinho se a conexão cair.

Grafana. Lê o PostgreSQL diretamente e vem com a fonte de dados e o painel já provisionados. São quatro gráficos, definidos em SQL (Apêndice E): horário de pico (observadores distintos por hora), densidade (posições por célula, ponderadas pela confiança), ocupação (observadores distintos por célula) e tempo de permanência por célula (tempo entre posições consecutivas que se moveram menos de 0,5 m).

3.8 Aplicativos móveis

O aplicativo iOS (Swift/SwiftUI) e o Android ("BlueGuide", Kotlin/Jetpack Compose) implementam os mesmos papéis de âncora e observador, com multilateração para qualquer $N \geq 3$ (Apêndice D), cadastro manual ou por realidade aumentada (ARKit/ARCore) da posição das âncoras, coletor de *fingerprints* em formato compatível com o do ESP32, e rastreamento ponto a ponto por UWB entre dois aparelhos (estilo AirTag).

3.9 Testes e verificação

A lógica pura de cada parte é coberta por testes automatizados que rodam sem *hardware*: Unity (via `pio test -e native`) no *firmware*, XCTest no iOS e JUnit no Android. A integração MQTT e os painéis do Grafana foram verificados manualmente contra um *broker* e um PostgreSQL reais (`docker compose up`), conferindo as linhas gravadas e a resposta da API do Grafana.

3.10 Divisão de tarefas

A Tabela 1 resume as contribuições, conforme o histórico do repositório.

Tabela 1. Divisão de tarefas no grupo.

Integrante	Principais contribuições
Anderson Gabriel	Aplicativo iOS inicial e rastreamento UWB; *firmware* ESP32 (*beacon* e coletor); script de âncora BLE no macOS.
Luís Filipe M. dos Santos	Troca de *tokens* UWB via BLE; correção dos testes de posicionamento; registro de dados no aplicativo.
Camila Cirne	Posicionamento BLE; *upload* de *fingerprints*; reorganização em *monorepo*; MQTT, Grafana e IPv6; aplicativo Android no *monorepo*.
[PENDENTE: nome]	**[PENDENTE: API e mapa *web* (repositório original `api_tracker_app`); revisar a tabela]**

4 Resultados

4.1 Recursos do *firmware*

A compilação para `esp32dev`, com a tabela de partições `huge_app` (necessária porque a pilha TLS do cliente HTTPS não cabe na partição padrão de 1,25 MB), ocupa **25,8% da RAM** e **51,3% da memória *flash*** disponíveis para a aplicação. O pacote de anúncio do *beacon* tem 31 bytes, exatamente o limite do anúncio legado, como relatado pela própria placa no evento `advertising` (`packet=31`).

4.2 Testes automatizados

A Tabela 2 resume os testes existentes no repositório.

Os cinco testes da estratégia de pilha dupla verificam que a conexão começa em IPv6, permanece nele antes do limite de 5 s, recai para IPv4 ao atingir o limite e não volta sozinha, tenta IPv6 de novo após um `reset()` e pula o IPv6 quando ele não está configurado.

4.3 Comunicação MQTT de ponta a ponta

O caminho ESP32 → Mosquitto → API → PostgreSQL foi validado com o *broker* real: as mensagens publicadas em `tracker/<node>/sightings` viraram li-

Tabela 2. Testes automatizados por componente.

Componente	Ferramenta	Casos de teste
Firmware ESP32 (17 suítes)	Unity / PlatformIO `native`	164
dos quais: estratégia IPv6/IPv4	Unity	5
Aplicativo iOS	XCTest	92
Aplicativo Android	JUnit (JVM)	45
API e mapa *web*	—	0 (validação manual)

nhas na tabela `sightings`, com a distância calculada pelo servidor. O mesmo teste pode ser repetido sem placa, publicando uma mensagem com `paho-mqtt` (`esp32-esp8266/USAGE.md`, cap. 8).

[PENDENTE: informar a taxa de mensagens MQTT por segundo observada e, se possível, a latência entre a leitura na serial e a linha no banco].

4.4 IPv6

A lógica de detecção de falha e retorno para IPv4 está implementada e testada. O caminho IPv6 em si **[PENDENTE: ainda não foi validado em *hardware*: nem o Mosquitto local usado até agora nem o Render oferecem IPv6. Executar o roteiro do `USAGE.md` (cap. 8) numa rede com IPv6 real e registrar aqui o log `mqtt_connected: connected over IPv6` e o teste de falha (desligar o IPv6 do *broker* e observar `mqtt_ipv6_fallback`)]**.

4.5 Painéis

Os quatro painéis do Grafana foram validados contra um PostgreSQL real, via `docker compose up` e a API do Grafana, e o mapa ao vivo recebe as posições por SSE à medida que chegam em `POST /fixes`.

[PENDENTE: inserir captura de tela do mapa ao vivo e do painel do Grafana com dados reais (`\includegraphics{figuras/mapa.png}`)]

Figura 4. Mapa ao vivo e painel do Grafana durante a demonstração.

4.6 Precisão do posicionamento

[PENDENTE: medir o erro de posição: com 3 a 4 *beacons* fixos em posições conhecidas, ficar em pelo menos 5 pontos marcados no chão, registrar a posição calculada

por 60 s em cada um e informar o erro médio e o máximo (em metros) numa tabela. Informar também a taxa de anúncios recebidos por segundo no coletor ESP32 e no celular (linhas `S` da serial e resumo do app), com e sem o Wi-Fi ligado no coletor].

5 Conclusão

O TrackerApp integra, num único repositório, *firmware* embarcado, aplicativos móveis, *backend* e painéis para posicionamento *indoor* por BLE. Os requisitos de comunicação MQTT, coleta e armazenamento de dados e visualização em tempo real foram atendidos e verificados com serviços reais. O uso de IPv6 foi implementado com detecção de falha e retorno automático para IPv4, com a lógica coberta por testes; falta a validação numa rede IPv6 real.

A separação entre lógica pura e adaptadores de *hardware* foi decisiva: permitiu testar no computador a maior parte do *firmware* (164 casos) e manter o mesmo protocolo de anúncio entre ESP32, iOS e Android.

Como trabalhos futuros, destacam-se: calibrar RSSI_0 e n no ambiente real; coletar o *dataset* de *fingerprints* e treinar modelos de regressão que comparem o desempenho com a multilateração; incorporar sensores e atuadores do kit (por exemplo, um LED RGB que indique o estado da conexão MQTT); adicionar autenticação e TLS ao *broker*; e substituir a chave única da API por credenciais por dispositivo.

O código-fonte, as instruções de uso e os diagramas estão disponíveis em https://github.com/agvmdsa/tracker-app-ios.

Referências

Bahl, P. e Padmanabhan, V. N. (2000). RADAR: an in-building RF-based user location and tracking system. In *Proceedings of IEEE INFOCOM 2000*, volume 2, p. 775–784.

Bluetooth SIG (2023). *Bluetooth Core Specification*, versão 5.4. Bluetooth Special Interest Group.

Deering, S. e Hinden, R. (2017). Internet Protocol, Version 6 (IPv6) Specification. RFC 8200, IETF.

Espressif Systems (2023). *ESP32 Series Datasheet*. Disponível em: https://www.espressif.com/en/support/documents/technical-documents.

Faragher, R. e Harle, R. (2015). Location fingerprinting with Bluetooth Low Energy beacons. *IEEE Journal on Selected Areas in Communications*, 33(11):2418–2428.

Light, R. A. (2017). Mosquitto: server and client implementation of the MQTT protocol. *Journal of Open Source Software*, 2(13):265.

OASIS (2014). *MQTT Version 3.1.1*. OASIS Standard. Editado por A. Banks e R. Gupta.

Ramírez, S. (2018). *FastAPI*. Disponível em: https://fastapi.tiangolo.com.

Rappaport, T. S. (2002). *Wireless Communications: Principles and Practice*. Prentice Hall, 2ª edição.

Schinazi, D. e Pauly, T. (2017). Happy Eyeballs Version 2: Better Connectivity Using Concurrency. RFC 8305, IETF.

Zafari, F., Gkelias, A. e Leung, K. K. (2019). A survey of indoor localization systems and technologies. *IEEE Communications Surveys & Tutorials*, 21(3):2568–2599.

Apêndices

Os trechos abaixo são cópias dos arquivos do repositório (https://github.com/agvmdsa/tracker-app-ios), com o caminho de cada um indicado na legenda.

A Estratégia de conexão IPv6/IPv4

Listagem 1. `esp32-esp8266/lib/domain/dual_stack_strategy.h`

```
// Decides which IP address family to attempt next for the MQTT broker connection, given how long
// the current attempt has run -- tries IPv6 first, falls back to IPv4 automatically if it doesn't
// connect within a timeout (the IPv6 "infra failure detection" requirement: the device notices
// the preferred path isn't there and recovers on its own, rather than just failing). Pure logic:
// no Arduino/network headers, so the fallback timing is tested on the host; the actual sockets
// are in hal/esp_mqtt_publisher.cpp.
#pragma once

#include <cstdint>

namespace domain {

enum class AddressFamily { V6, V4 };

class DualStackConnectStrategy {
public:
    // `v6TimeoutMs`: how long to let an IPv6 attempt run before giving up on it for this attempt
    // and switching to IPv4. `hasV6`: false when no IPv6 address is configured at all -- then
    // every attempt goes straight to IPv4, skipping the IPv6 phase entirely.
    DualStackConnectStrategy(uint32_t v6TimeoutMs, bool hasV6) : v6TimeoutMs_(v6TimeoutMs), hasV6_(hasV6) {}

    // Call once when starting a fresh connection attempt (e.g. after Wi-Fi comes up, or after a
    // previous connection was lost) -- always retries IPv6 first, since a disconnect doesn't mean
    // IPv6 itself is gone, just that this one connection was.
    void reset(uint32_t nowMs) {
        family_ = hasV6_ ? AddressFamily::V6 : AddressFamily::V4;
        attemptStartMs_ = nowMs;
    }

    // Call periodically while not yet connected. Returns the family that should be in use right
    // now; switches itself from V6 to V4 once v6TimeoutMs_ has passed without connecting.
    AddressFamily poll(uint32_t nowMs) {
        if (family_ == AddressFamily::V6 && (nowMs - attemptStartMs_) >= v6TimeoutMs_) {
            family_ = AddressFamily::V4;
            attemptStartMs_ = nowMs;
        }
        return family_;
    }

    AddressFamily currentFamily() const { return family_; }

private:
    uint32_t v6TimeoutMs_;
    bool hasV6_;
    uint32_t attemptStartMs_ = 0;
```

```
    AddressFamily family_ = AddressFamily::V4;
};

}  // namespace domain
```

Listagem 2. Testes da estratégia (`test_dual_stack_strategy/test_main.cpp`)

```
// IPv6-first, fall-back-to-IPv4 timing for the MQTT connection attempt.
#include <unity.h>

#include "dual_stack_strategy.h"

using namespace domain;

void setUp() {}
void tearDown() {}

static void test_starts_on_v6_when_v6_is_configured() {
    DualStackConnectStrategy s(5000, /*hasV6=*/true);
    s.reset(1000);
    TEST_ASSERT_TRUE(AddressFamily::V6 == s.poll(1000));
}

static void test_stays_on_v6_before_the_timeout() {
    DualStackConnectStrategy s(5000, /*hasV6=*/true);
    s.reset(1000);
    TEST_ASSERT_TRUE(AddressFamily::V6 == s.poll(1000));
    TEST_ASSERT_TRUE(AddressFamily::V6 == s.poll(4000));
    TEST_ASSERT_TRUE(AddressFamily::V6 == s.poll(5999));
}

static void test_falls_back_to_v4_once_the_timeout_elapses() {
    DualStackConnectStrategy s(5000, /*hasV6=*/true);
    s.reset(1000);
    TEST_ASSERT_TRUE(AddressFamily::V4 == s.poll(6000));
    // and stays there, it doesn't bounce back to V6 on its own
    TEST_ASSERT_TRUE(AddressFamily::V4 == s.poll(60000));
}

static void test_a_fresh_reset_tries_v6_again_even_after_a_previous_fallback() {
    DualStackConnectStrategy s(5000, /*hasV6=*/true);
    s.reset(1000);
    s.poll(10000);  // falls back to V4
    TEST_ASSERT_TRUE(AddressFamily::V4 == s.currentFamily());
    s.reset(20000);  // e.g. Wi-Fi reconnected, or the MQTT connection dropped
    TEST_ASSERT_TRUE(AddressFamily::V6 == s.poll(20000));
}

static void test_skips_v6_entirely_when_none_is_configured() {
    DualStackConnectStrategy s(5000, /*hasV6=*/false);
    s.reset(1000);
    TEST_ASSERT_TRUE(AddressFamily::V4 == s.poll(1000));
    TEST_ASSERT_TRUE(AddressFamily::V4 == s.poll(1500));  // never waits for a timeout
     that won't matter
}

int main(int, char**) {
    UNITY_BEGIN();
    RUN_TEST(test_starts_on_v6_when_v6_is_configured);
    RUN_TEST(test_stays_on_v6_before_the_timeout);
    RUN_TEST(test_falls_back_to_v4_once_the_timeout_elapses);
    RUN_TEST(test_a_fresh_reset_tries_v6_again_even_after_a_previous_fallback);
    RUN_TEST(test_skips_v6_entirely_when_none_is_configured);
    return UNITY_END();
}
```

B Publicador MQTT no ESP32

Listagem 3. esp32-esp8266/src/hal/esp_mqtt_publisher.cpp

```cpp
#include "hal/esp_mqtt_publisher.h"

#include <WiFi.h>

#include <cstring>

#include "sighting_payload.h"

namespace {

void copyString(char* destination, std::size_t capacity, const char* source) {
    std::strncpy(destination, source, capacity - 1);
    destination[capacity - 1] = '\0';
}

}  // namespace

void EspMqttPublisher::begin(const char* wifiSsid, const char* wifiPassword, const char* host,
                             const char* hostV6, uint16_t port, const char* user, const char* password,
                             const char* topicPrefix, const char* nodeId) {
    copyString(wifiSsid_, sizeof(wifiSsid_), wifiSsid);
    copyString(wifiPassword_, sizeof(wifiPassword_), wifiPassword);
    enabled_ = host[0] != '\0';
    if (!enabled_) return;

    copyString(host_, sizeof(host_), host);
    copyString(hostV6_, sizeof(hostV6_), hostV6);
    port_ = port;
    strategy_ = domain::DualStackConnectStrategy(5000, hostV6_[0] != '\0');
    std::snprintf(topic_, sizeof(topic_), "%s/%s/sightings", topicPrefix, nodeId);
    if (user[0] != '\0') client_.setCredentials(user, password);
    if (hostV6_[0] != '\0') {
        WiFi.enableIPv6();
        events_.event("info", "mqtt_dual_stack", "IPv6 configured, will be tried before IPv4 on each connection attempt");
    }
}

bool EspMqttPublisher::publishSighting(const char* node, const char* beacon, int rssi) {
    if (!enabled_ || !client_.connected()) return false;
    char body[128];
    if (domain::formatSightingPayload(body, sizeof(body), node, beacon, rssi) < 0) return false;
    return client_.publish(topic_, 0, false, body) != 0;
}

void EspMqttPublisher::poll() {
    if (!enabled_) return;

    connectWifiIfNeeded();
    client_.loop();

    const bool wifiConnected = WiFi.status() == WL_CONNECTED;
    if (!wifiConnected) {
        wifiWasConnected_ = false;
        return;
    }
    if (!wifiWasConnected_) {
        // Wi-Fi just came up: start a fresh attempt, trying IPv6 first again if it's configured
        // -- the timeout clock starts now, not whenever Wi-Fi itself happened to begin connecting.
        wifiWasConnected_ = true;
        strategy_.reset(millis());
        connectingFamily_ = strategy_.currentFamily();
        connectToCurrentFamily();
```

```
        return;
    }

    if (client_.connected()) {
        if (!mqttWasConnected_) {
            mqttWasConnected_ = true;
            events_.event("info", "mqtt_connected",
                          connectingFamily_ == domain::AddressFamily::V6 ? "connected over IPv6" : "connected over IPv4");
        }
        return;
    }
    mqttWasConnected_ = false;

    const domain::AddressFamily wanted = strategy_.poll(millis());
    if (wanted != connectingFamily_) {
        connectingFamily_ = wanted;
        if (wanted == domain::AddressFamily::V4) {
            events_.event("warn", "mqtt_ipv6_fallback", "IPv6 did not connect in time, falling back to IPv4");
        }
        connectToCurrentFamily();
    } else if (client_.disconnected()) {
        connectToCurrentFamily();
    }
}

void EspMqttPublisher::connectToCurrentFamily() {
    const char* host = connectingFamily_ == domain::AddressFamily::V6 ? hostV6_ : host_;
    client_.setServer(host, port_);
    client_.connect();
}

void EspMqttPublisher::connectWifiIfNeeded() {
    if (WiFi.status() == WL_CONNECTED) return;
    if (WiFi.status() != WL_DISCONNECTED) return;  // already trying
    WiFi.begin(wifiSsid_, wifiPassword_);
}
```

Listagem 4. Carga JSON da mensagem (`lib/domain/sighting_payload.cpp`)

```
#include "sighting_payload.h"

#include <cstdio>

namespace domain {

int formatSightingPayload(char* buf, std::size_t cap, const char* node, const char* beacon, int rssi) {
    const int written = std::snprintf(
        buf, cap, "{\"observerID\":\"%s\",\"anchorID\":\"%s\",\"rssi\":%d}", node, beacon, rssi);
    if (written < 0 || static_cast<std::size_t>(written) >= cap) return -1;
    return written;
}

}  // namespace domain
```

C Assinante MQTT e modelo de perda de percurso (API)

Listagem 5. `applications/api/app/mqtt_consumer.py`

```
"""Subscribes to the ESP32 collectors' live RSSI telemetry over MQTT and persists it the same
way `POST /sightings` does. This is the project's IoT/pub-sub data path -- distinct from the
```

```
HTTP-only paths: `/sightings`/`/fixes` are phones reporting computed results, `/fingerprints` is
a one-shot batch upload after a finished recording, and this is the ESP32 collector's continuous,
unlabelled telemetry (the same data its `L`/idle-view serial lines carry), which a pub/sub broker
fits better than a request per reading would.

Optional: does nothing if `MQTT_HOST` isn't set, same convention `API_KEY` does NOT follow (that
one's required) -- MQTT is additive telemetry, not core to the API working, so a missing broker
shouldn't block startup or break the rest of the API.
"""

import json
import logging
import os
from datetime import datetime, timezone

import paho.mqtt.client as mqtt

from app.database import SessionLocal
from app.models import Sighting
from app.path_loss import distance_meters

logger = logging.getLogger("mqtt_consumer")

MQTT_HOST = os.environ.get("MQTT_HOST", "")
MQTT_PORT = int(os.environ.get("MQTT_PORT", "1883"))
# Matches embedded/include/config.example.h's TOPIC_PREFIX default ("tracker") and the node-scoped
# shape a collector would publish under, e.g. tracker/A1B2C3/sightings.
MQTT_TOPIC = os.environ.get("MQTT_TOPIC", "tracker/+/sightings")


def _insert_sighting(payload: dict) -> None:
    """One JSON object per MQTT message, same field names/shapes as `SightingIn` (see
    app/schemas.py) -- camelCase, not the serial protocol's CSV-ish `R`/`L` line format. A
    malformed message is logged and dropped, not raised: one bad reading from one board
    shouldn't take down the subscriber thread.

    `seenAt` is optional, same reasoning as `FingerprintRecordIn.seenAt`: the ESP32 has no
    battery-backed, NTP-synced clock by default, so it may not send one -- falls back to the
    received-at time when absent.
    """
    received_at = datetime.now(timezone.utc)
    seen_at_field = payload.get("seenAt")
    seen_at = datetime.fromisoformat(seen_at_field.replace("Z", "+00:00")) if seen_at_field else received_at
    db = SessionLocal()
    try:
        db.add(
            Sighting(
                observer_id=payload["observerID"],
                anchor_id=payload["anchorID"],
                rssi=payload["rssi"],
                distance_meters=distance_meters(payload["rssi"]),
                seen_at=seen_at,
                received_at=received_at,
                sample_count=payload.get("sampleCount"),
                rssi_std_dev=payload.get("rssiStdDev"),
                device_model=payload.get("deviceModel"),
            )
        )
        db.commit()
```

```
    finally:
        db.close()


def _on_connect(client: mqtt.Client, userdata: object, flags: dict, reason_code: object, properties: object = None) -> None:
    logger.info("connected to MQTT broker %s:%s, subscribing to %s", MQTT_HOST, MQTT_PORT, MQTT_TOPIC)
    client.subscribe(MQTT_TOPIC)


def _on_message(client: mqtt.Client, userdata: object, msg: mqtt.MQTTMessage) -> None:
    try:
        payload = json.loads(msg.payload.decode("utf-8"))
        _insert_sighting(payload)
    except Exception:
        logger.exception("dropping unprocessable MQTT message on %s", msg.topic)


def start_mqtt_consumer() -> mqtt.Client | None:
    """Starts the subscriber in a background thread (`loop_start`) and returns the client, or
    None when `MQTT_HOST` isn't set. `connect_async` + `loop_start` retry the connection on their
    own if the broker isn't reachable yet -- this never blocks or raises on startup.
    """
    if not MQTT_HOST:
        logger.info("MQTT_HOST not set, MQTT telemetry consumer disabled")
        return None
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = _on_connect
    client.on_message = _on_message
    client.connect_async(MQTT_HOST, MQTT_PORT, keepalive=60)
    client.loop_start()
    return client


def stop_mqtt_consumer(client: mqtt.Client | None) -> None:
    if client is None:
        return
    client.loop_stop()
    client.disconnect()
```

Listagem 6. `applications/api/app/path_loss.py`

```
REFERENCE_RSSI = -59.0
PATH_LOSS_EXPONENT = 2.0


def distance_meters(rssi: float) -> float:
    return 10.0 ** ((REFERENCE_RSSI - rssi) / (10.0 * PATH_LOSS_EXPONENT))
```

D Multilateração (iOS)

Listagem 7. `Multilateration.swift` (`applications/ios/.../Positioning/`)

```
import Foundation

struct AnchorRange {
    let anchor: Anchor
    let distanceMeters: Double
}

struct ObserverFix {
    let readings: [AnchorRange]
    let position: Point2D
    let uncertaintyMeters: Double
```

```
}

enum Multilateration {
    private static let epsilon = 1e-9

    static func solve(readings: [AnchorRange]) -> ObserverFix? {
        guard readings.count >= 3 else { return nil }

        let first = readings[0]
        let p1 = first.anchor.position
        let d1 = first.distanceMeters

        var m11 = 0.0, m12 = 0.0, m22 = 0.0
        var c1 = 0.0, c2 = 0.0

        for reading in readings.dropFirst() {
            let p = reading.anchor.position
            let a1 = 2 * (p.x - p1.x)
            let a2 = 2 * (p.y - p1.y)
            let b = sq(d1) - sq(reading.distanceMeters) + sq(p.x) - sq(p1.x) + sq(p.y) - sq(p1.y)

            m11 += a1 * a1
            m12 += a1 * a2
            m22 += a2 * a2
            c1 += a1 * b
            c2 += a2 * b
        }

        let determinant = m11 * m22 - m12 * m12
        guard abs(determinant) > epsilon else { return nil }

        let position = Point2D(
            x: (c1 * m22 - m12 * c2) / determinant,
            y: (m11 * c2 - c1 * m12) / determinant
        )

        let residual = sqrt(
            readings.reduce(0.0) { partial, reading in
                partial + sq(reading.anchor.position.distance(to: position) - reading.distanceMeters)
            } / Double(readings.count)
        )

        return ObserverFix(readings: readings, position: position, uncertaintyMeters: residual)
    }

    private static func sq(_ value: Double) -> Double { value * value }
}
```

E Consultas dos painéis do Grafana

Listagem 8. `applications/api/grafana/queries.sql`

```
-- Grafana panel queries for the Positioning API's `fixes` table.
-- Paste the SELECT directly into a Grafana panel's "Code" query editor (Postgres datasource).
-- $__timeFilter(recorded_at) is a Grafana macro -- it expands to a WHERE clause using the
-- dashboard's time range picker, so these queries respect whatever window you select in Grafana.

-- ============================================================
-- 1. Heatmap density -- grid cells weighted by fix confidence
-- ============================================================
-- Panel type: Heatmap (or Geomap with a static floor-plan image as the base layer, x/y as
```

```sql
-- lon/lat-like coordinates). 0.5 is the grid cell size in metres -- shrink it for a sharper
-- heatmap once there's enough data density to support it, grow it if cells look too sparse.
-- `greatest(uncertainty_meters, 0.1)` floors the weight so a near-zero uncertainty fix doesn't
-- dominate the whole map by itself.
SELECT
    round(x / 0.5) * 0.5 AS grid_x,
    round(y / 0.5) * 0.5 AS grid_y,
    count(*) AS fix_count,
    sum(1.0 / greatest(uncertainty_meters, 0.1)) AS weighted_density
FROM fixes
WHERE $__timeFilter(recorded_at)
GROUP BY grid_x, grid_y
ORDER BY weighted_density DESC;

-- ============================================================
-- 2. Occupancy -- distinct observers seen per grid cell
-- ============================================================
-- Panel type: Heatmap or a table. This is the "how many different phones were in this area"
-- view -- counts distinct observer_id, not raw fix count, so one phone sending fixes every
-- second doesn't look like a crowd.
SELECT
    round(x / 0.5) * 0.5 AS grid_x,
    round(y / 0.5) * 0.5 AS grid_y,
    count(DISTINCT observer_id) AS distinct_observers
FROM fixes
WHERE $__timeFilter(recorded_at)
GROUP BY grid_x, grid_y
ORDER BY distinct_observers DESC;

-- ============================================================
-- 3. Peak hours -- traffic over time (not spatial)
-- ============================================================
-- Panel type: Time series. date_trunc('hour', ...) buckets into hourly columns; swap for
-- 'day'/'minute' depending on how long a time range the dashboard is set to.
SELECT
    date_trunc('hour', recorded_at) AS time,
    count(DISTINCT observer_id) AS distinct_observers,
    count(*) AS fix_count
FROM fixes
WHERE $__timeFilter(recorded_at)
GROUP BY time
ORDER BY time;

-- ============================================================
-- 4. Dwell time per cell -- "stopped here" vs "walked through"
-- ============================================================
-- Panel type: Heatmap or table. No new phone data needed for this -- it's inferred purely from
-- consecutive fixes already being sent: if an observer's position barely moves between two
-- fixes (< 0.5m), the elapsed time between them counts as "dwelling" at that cell. A phone that
-- jumps between distant fixes contributes nothing (it was just passing through).
WITH ordered_fixes AS (
    SELECT
        observer_id,
        x,
        y,
        recorded_at,
        lag(x) OVER (PARTITION BY observer_id ORDER BY recorded_at) AS prev_x,
        lag(y) OVER (PARTITION BY observer_id ORDER BY recorded_at) AS prev_y,
        lag(recorded_at) OVER (PARTITION BY observer_id ORDER BY recorded_at) AS prev_time
    FROM fixes
```

```
    WHERE $__timeFilter(recorded_at)
),
movement AS (
    SELECT
        x,
        y,
        sqrt(power(x - prev_x, 2) + power(y - prev_y, 2)) AS distance_moved,
        extract(epoch FROM (recorded_at - prev_time)) AS seconds_elapsed
    FROM ordered_fixes
    WHERE prev_x IS NOT NULL
)
SELECT
    round(x / 0.5) * 0.5 AS grid_x,
    round(y / 0.5) * 0.5 AS grid_y,
    sum(CASE WHEN distance_moved < 0.5 THEN seconds_elapsed ELSE 0 END) AS
    dwell_seconds
FROM movement
GROUP BY grid_x, grid_y
ORDER BY dwell_seconds DESC;
```

F Infraestrutura

Listagem 9. `applications/api/docker-compose.yml`

```
services:
  postgres:
    image: postgres:16
    restart: unless-stopped
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: positioning
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 5s
      retries: 10

  mosquitto:
    image: eclipse-mosquitto:2.0
    restart: unless-stopped
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
    ports:
      - "1883:1883"

  api:
    build: .
    restart: unless-stopped
    environment:
      DATABASE_URL: postgresql+psycopg://postgres:postgres@postgres:5432/positioning
      API_KEY: dev-only-local-key
      MQTT_HOST: mosquitto
    ports:
      - "8000:8000"
    depends_on:
      postgres:
        condition: service_healthy
      mosquitto:
        condition: service_started

  frontend:
    # The live floor-plan map web app (Next.js, applications/web -- a sibling of this
    directory
```

```
    # now that the repo is a monorepo, so the build context is "../web" not "./front")
    .
    # Browser-facing, so NEXT_PUBLIC_API_BASE_URL points at the host-published API
    port, not the
    # internal "api" service name. Mapped to 3001 since Grafana owns 3000.
    build:
      context: ../web
      args:
        NEXT_PUBLIC_API_BASE_URL: http://localhost:8000
        NEXT_PUBLIC_USE_MOCK: "false"
        NEXT_PUBLIC_POLL_INTERVAL_MS: "1000"
    restart: unless-stopped
    ports:
      - "3001:3000"
    depends_on:
      - api

  grafana:
    image: grafana/grafana:11.2.0
    restart: unless-stopped
    environment:
      GF_SECURITY_ADMIN_USER: admin
      GF_SECURITY_ADMIN_PASSWORD: admin
      POSTGRES_HOST: postgres
      POSTGRES_PORT: "5432"
      POSTGRES_DB: positioning
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_SSLMODE: disable
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
      - ./grafana/provisioning:/etc/grafana/provisioning
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  postgres_data:
  grafana_data:
```

Listagem 10. `applications/api/mosquitto/mosquitto.conf`

```
# Local dev only: anonymous, unencrypted. Eclipse Mosquitto 2.x refuses all
    connections unless a
# listener is explicitly configured, so this exists even though the settings
    themselves are
# permissive. Don't deploy this config as-is anywhere reachable from the internet.
listener 1883
allow_anonymous true
```